

Bericht

Jiaqi Wang

24. Juli 2026

1 Entwurf der Grundplatte mit Inkscape

Um den Schrittmotor, die Steuerplatine und den Akku zu einer kompakten Einheit zusammenzufassen, wurde eine Grundplatte mit der Vektorgrafiksoftware *Inkscape* entworfen. Ziel war es, alle Komponenten sicher auf einer gemeinsamen Trägerplatte zu befestigen und den späteren Aufbau zu vereinfachen.

Zunächst wurden mit einer Schieblehre die Positionen der Befestigungsbohrungen vermessen. Insgesamt wurden vier Bohrungen vorgesehen. Für den Schrittmotor wurden zwei Bohrungen mit einem Durchmesser von **M5** erstellt, während für die Steuerplatine zwei Bohrungen mit einem Durchmesser von **M3** vorgesehen wurden.

Die endgültige Größe der Grundplatte beträgt **31 cm × 20 cm**. Um die Anforderungen der Laserschneidmaschine zu erfüllen, wurden sämtliche Konturen ausschließlich aus roten Linien mit einer Linienbreite von **0,03 mm** erstellt. Alle Rechtecke und Kreise besitzen keine Flächenfüllung (*Fill: None*), sodass ausschließlich die Schneidkonturen exportiert werden.

Nach Abschluss des Entwurfs wurde die Zeichnung im SVG-Format unter dem Dateinamen `base_v10.svg` gespeichert und anschließend für den Laserschneidprozess verwendet.

2 Mehrere TCP-Clients

In der ersten Version unterstützte der ESP32 nur einen TCP-Client. Für das Projekt sollten jedoch sowohl die ROS 2-Anwendung als auch die PC-Software gleichzeitig mit dem ESP32 kommunizieren. Daher wurde der TCP-Server auf Mehrbenutzerbetrieb erweitert.

Die maximale Anzahl der gleichzeitig verbundenen Clients wird über ein Array von `WiFiClient` verwaltet.

```
#define MAX_TCP_CLIENTS 4

WiFiClient tcpClients[MAX_TCP_CLIENTS];
String tcpRxBuffers[MAX_TCP_CLIENTS];
```

Neue Verbindungen werden automatisch einem freien Slot zugewiesen.

```
int slot = findFreeTcpClientSlot();

if (slot >= 0) {
    tcpClients[slot] = newClient;
    tcpRxBuffers[slot] = "";
}
```

Dadurch können beispielsweise die ROS 2-Steuerung und die PC-Benutzeroberfläche gleichzeitig Befehle senden und Statusinformationen vom ESP32 empfangen.

3 Motorsteuerung mit ROS 2

Nach der Verbindung mit dem WLAN-Hotspot des ESP32 stellt ein ROS 2-Node eine TCP-Verbindung zum ESP32 her. Befehle werden über das Topic `/esp32/command` empfangen und als Textnachrichten an den ESP32 weitergeleitet.

```
self.command_subscription = self.create_subscription(
    String,
    '/esp32/command',
    self.command_callback,
    10
)

packet = (command + '\n').encode('utf-8')
current_socket.sendall(packet)
```

Die Rückmeldungen des ESP32 werden über die Topics `/esp32/response` und `/esp32/motor_state` veröffentlicht.

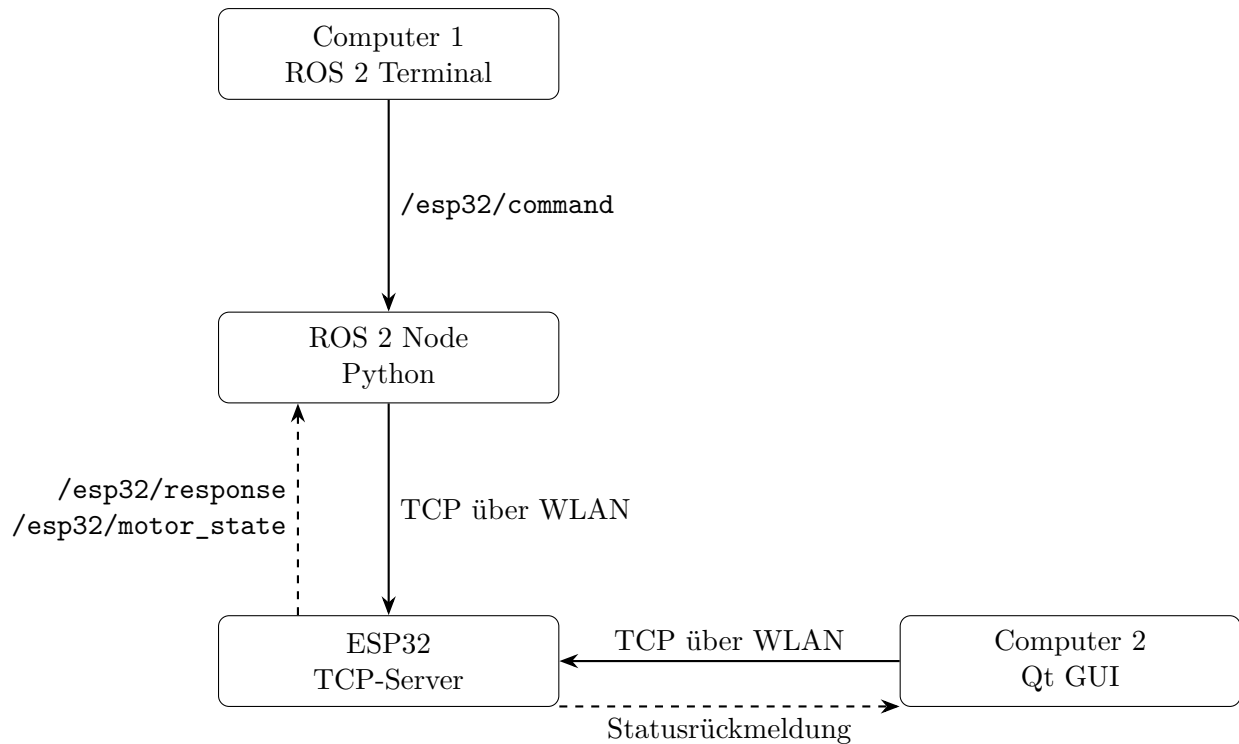


Abbildung 1: Kommunikationsstruktur zwischen ROS 2, Qt GUI und ESP32.

Typische Steuerbefehle sind:

```

# Rckmeldungen anzeigen
ros2 topic echo /esp32/response
ros2 topic echo /esp32/motor_state

# Motor bewegen
ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'ANGLE:30'}"

ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'ANGLE:-20'}"

# Geschwindigkeit einstellen
ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'SPEED:100'}"

ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'SPEED:300'}"

# Motor stoppen / fortsetzen
ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'STOP'}"

ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'RESUME'}"

# Not-Aus / Not-Aus aufheben

```

```

ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'ESTOP'}"

ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'ESTOP_CLEAR'}"

# Motorstatus abfragen
ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'MOTOR_STATE?'}"

# LED ein / aus
ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'LED_ON'}"

ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'LED_OFF'}"

# LED-Farbe einstellen
ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'RGB:255,0,0'}"

ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'RGB:0,255,0'}"

ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'RGB:0,0,255'}"

ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'RGB:255,255,255'}"

ros2 topic pub --once /esp32/command \
std_msgs/msg/String "{data:'RGB:0,0,0'}"

```

4 TO-DO-List

In Zukunft soll der aktuell separate ROS 2-Motorsteuerungsknoten in den Hauptkommunikationsknoten integriert werden. Dadurch werden die Kommunikation mit dem ESP32 sowie die Motorsteuerung zentral in einem ROS 2-Knoten verwaltet.